LAPORAN PRAKTIKUM
POLYMORPHISM

Dibuat untuk memenuhi salah satu tugas Praktikum Mata Kuliah Pengembangan Berorientasi Objek



Dibuat Oleh:

NAMA	: Rena Fiantina
NIM	: 251511028
KELAS	: 2-A
PROGRAM STUDI	: D3 Teknik Informatika

TEKNIK KOMPUTER DAN INFORMATIKA
POLITEKNIK NEGERI BANDUNG
2026

A. Praktikum I - Another Type of Employee

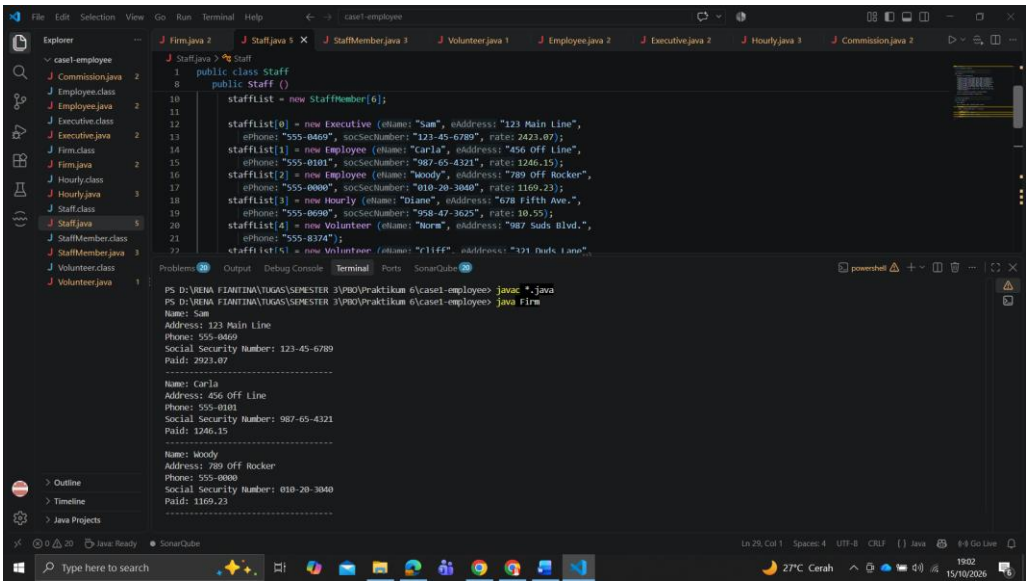
a. Pemetaan Hierarchy dan Reference Type

Class StaffMember merupakan superclass dengan Class Volunteer dan Employee yang menjadi subclass nya, sedangkan Executive dan Hourly merupakan subclass dari Employee.

Dari source code, StaffMember merupakan abstract class karena dideklarasikan menggunakan keyword abstract dan memiliki method abstrak pay(). Sementara itu, Volunteer, Employee, Executive, dan Hourly merupakan concrete class yang dapat dibuat menjadi objek.

Checkpoint 1 : *Apa tipe array pada Staff? Mengapa array tersebut dapat berisi Executive, Employee, Hourly, Volunteer, dan nanti Commission?*

⇨ Tipe array yang digunakan pada class Staff adalah StaffMember[]. Array tersebut dapat menyimpan objek Executive, Employee, Hourly, dan Volunteer karena kelas-kelas tersebut merupakan subclass dari StaffMember. Setelah ditambahkan, Commission juga dapat disimpan dalam array tersebut karena merupakan subclass dari Hourly, yang akhirnya mewarisi StaffMember.



Checkpoint 2 : *Reference yang dipakai oleh loop adalah StaffMember, tetapi implementation pay() yang berjalan tidak selalu StaffMember. Apa yang menentukan implementation yang dipilih?*

⇨ Implementasi method pay() yang dipilih ditentukan oleh jenis objek sebenarnya (runtime type) yang dirujuk oleh variabel StaffMember. Meskipun reference yang digunakan bertipe StaffMember, ketika method pay() dipanggil, Java menjalankan implementasi yang sesuai dengan kelas objek tersebut. Mekanisme ini disebut dynamic binding dan merupakan salah satu penerapan polymorphism.

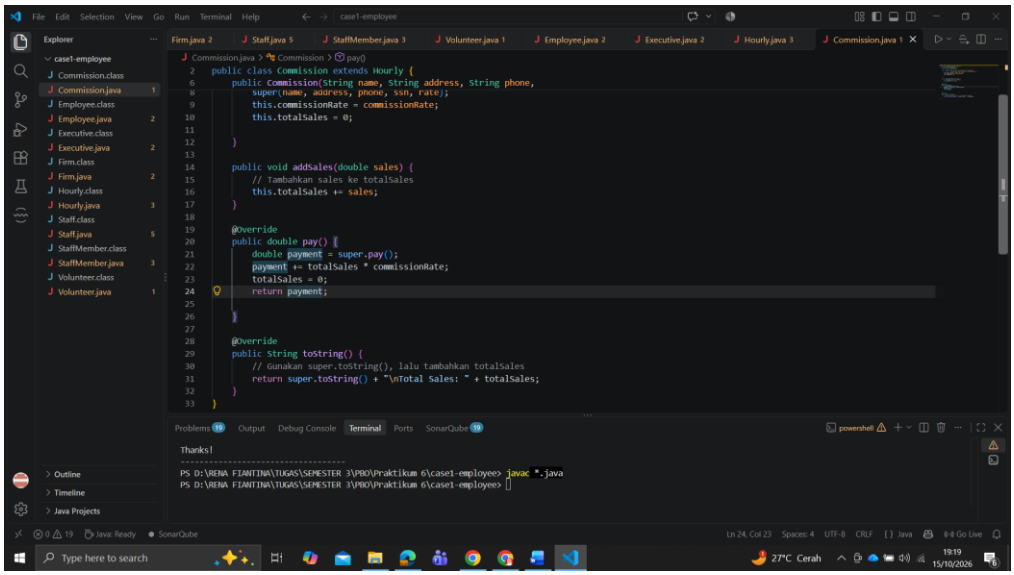
Prediksi Singkat Step 3

1. Tulis state baru yang hanya dibutuhkan Commission.
   - ⇨ totalSales bertipe double, untuk menyimpan total penjualan yang dilakukan karyawan.
   - ⇨ commissionRate bertipe double, untuk menyimpan persentase komisi dalam bentuk desimal.

2. Behavior Hourly apa yang masih dibutuhkan Commission?
   - ⇨ pencatatan jam kerja melalui addHours() dan perhitungan upah berdasarkan jam kerja melalui pay()

3. Apakah Commission perlu menulis ulang mekanisme perhitungan upah per jam dari nol? Jelaskan prediksi Anda.
   - ⇨ Tidak. Commission dapat menggunakan kembali perhitungan upah per jam dari class Hourly dengan super.pay(), kemudian menambahkan hasil komisi penjualan.

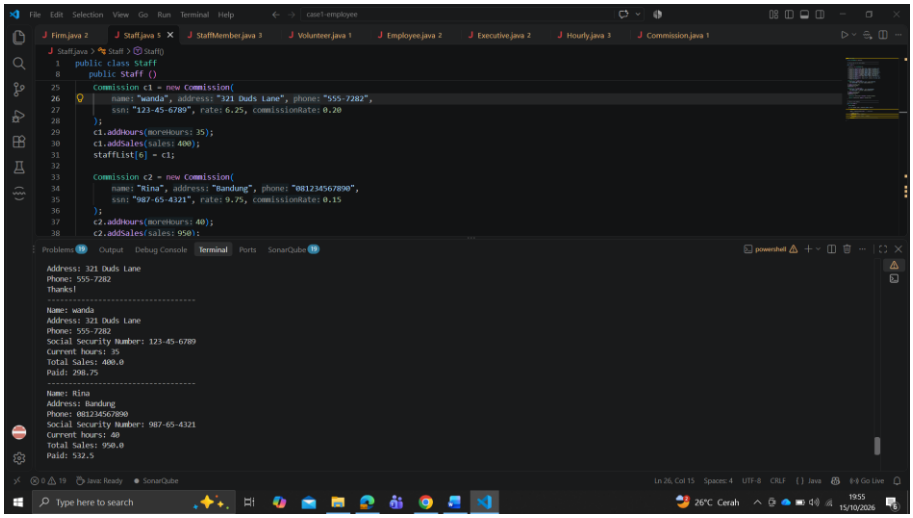
b. Implementasi Class Commission



Checkpoint 3 : *Mengapa pay() pada Commission sebaiknya memanggil super.pay() daripada menghitung kembali hoursWorked? Hubungkan dengan encapsulation dan reuse behavior.*

- ⇨ Method pay() pada class Commission sebaiknya memanggil super.pay() karena perhitungan upah berdasarkan jam kerja sudah tersedia pada class Hourly. Dengan demikian, Commission tidak perlu menghitung ulang hoursWorked dari awal.

Tambahkan dua Commission :

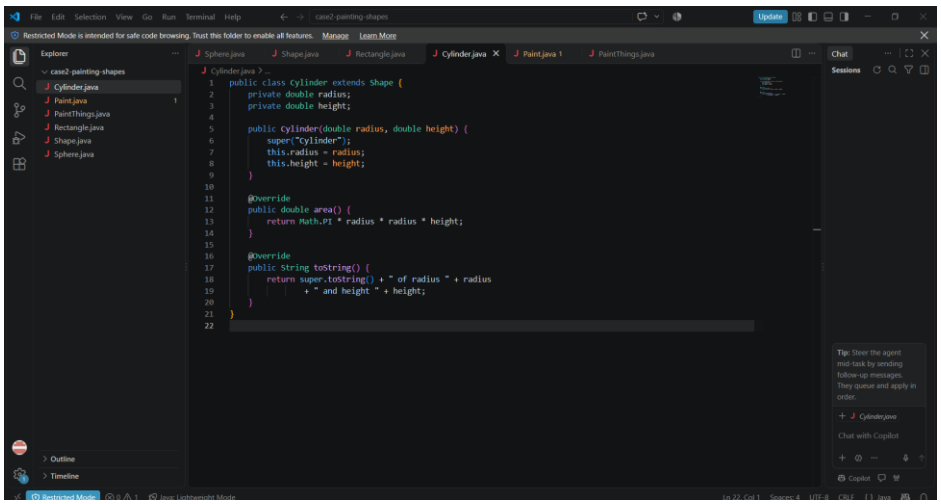


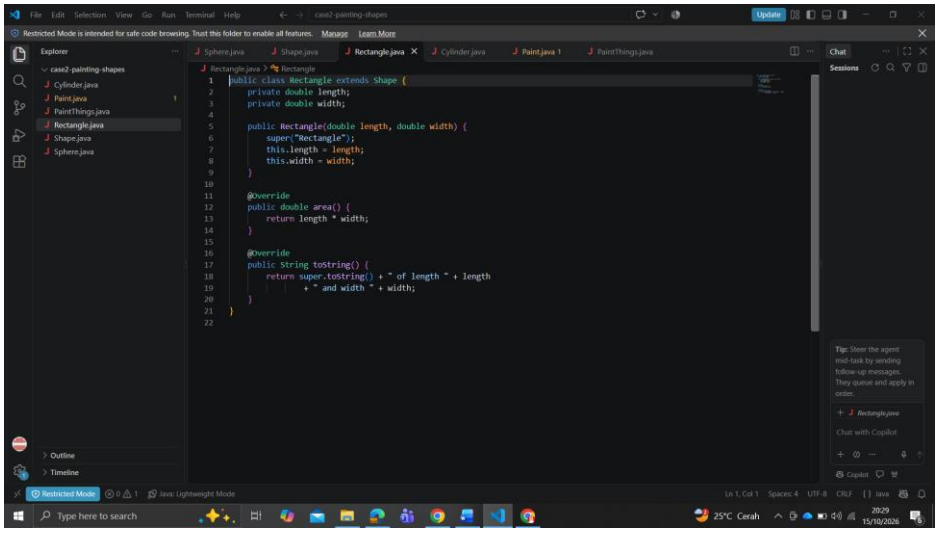
B. Praktikum II - Painting Shapes

a. Pembuatan Abstract Class Shape & Subclass Sphere, Rectangle, dan Cylinder

1. Prediksi singkat: apakah new Shape("X") dapat dibuat?
   new Shape("X") tidak dapat dibuat karena Shape merupakan abstract class. Abstract class tidak dapat diinstansiasi secara langsung. Objek harus dibuat dari subclass konkret, seperti Sphere, Rectangle, atau Cylinder

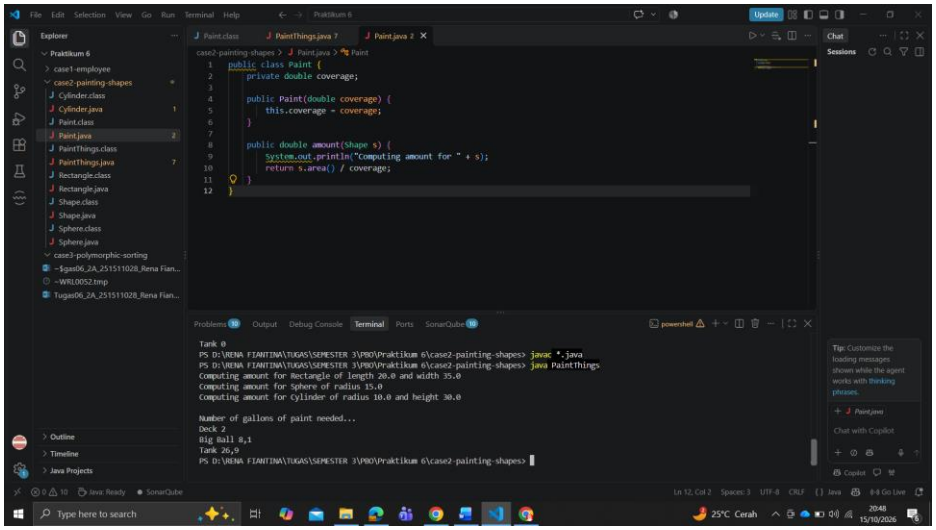
2. Jelaskan mengapa area() dideklarasikan tetapi tidak diberi body di Shape.
   Method area() dideklarasikan tanpa body karena setiap bentuk memiliki rumus perhitungan luas yang berbeda. Dengan menjadikannya abstract method, setiap subclass diwajibkan mengimplementasikan method area() sesuai karakteristik bentuknya masing-masing.





b. Implementasi Method Paint.amount(Shape)

Mengimplementasikan method amount() pada class Paint untuk menghitung kebutuhan cat berdasarkan luas objek dan kapasitas cat yang bisa menutupi area area.



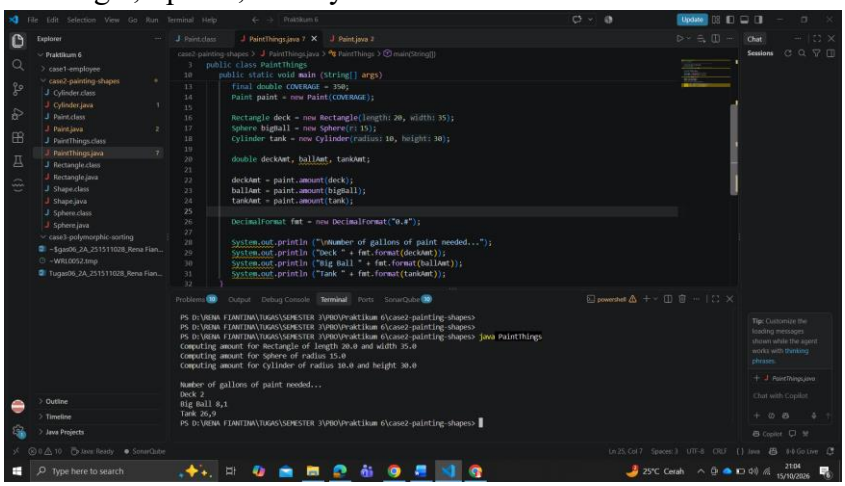
Program berhasil menghitung kebutuhan cat untuk objek Rectangle, Sphere, dan Cylinder. Hasil yang diperoleh adalah Deck sebanyak 2, Big Ball sebanyak 8,1 dan Tank sebanyak 26,9.

Checkpoint 5 : *Mengapa Paint.amount(Shape s) dapat bekerja untuk tiga subclass yang rumus area()-nya berbeda?*

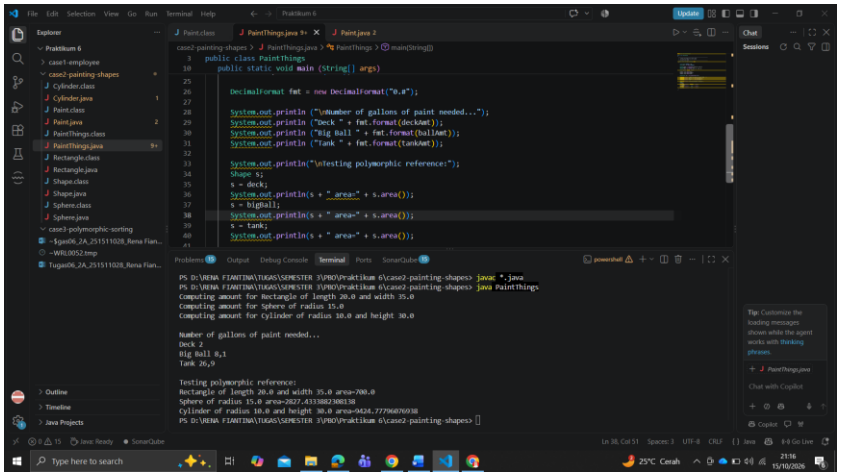
⇨ Karena method amount() menerima parameter bertipe Shape dan memanggil s.area(). Meskipun reference-nya bertipe Shape, implementasi area() yang dijalankan mengikuti objek sebenarnya, yaitu Rectangle, Sphere, atau Cylinder. Mekanisme ini disebut polymorphism melalui method overriding dan dynamic binding. Karena itu, Paint tidak membutuhkan if-else atau switch untuk membedakan jenis bentuk.

c. Pengujian PaintThings & Polymorphic Reference

Menguji perhitungan kebutuhan cat dan penerapan polymorphic reference menggunakan satu variabel bertipe Shape yang merujuk secara bergantian ke objek Rectangle, Sphere, dan Cylinder.



Pengujian Polymorphic reference

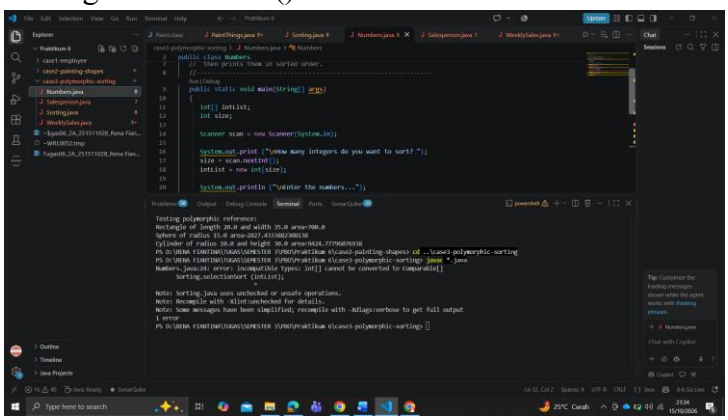


Program berhasil menampilkan luas Rectangle sebesar 700, Sphere sebesar 2827,43, dan Cylinder sebesar 9424,78. Hasil tersebut menunjukkan bahwa pemanggilan method area() menjalankan implementasi sesuai dengan objek yang dirujuk.

C. Praktikum III - Polymorphic Sorting

a. Pengujian Error pada int[]

Menguji penggunaan array bertipe int[] sebagai parameter method Sorting.selectionSort()



Saat program dikompilasi, compiler menampilkan pesan error yang menunjukkan bahwa argumen bertipe int[] tidak sesuai dengan parameter yang dibutuhkan oleh method Sorting.selectionSort().

Perbandingan parameter selectionSort dengan tipe int[]

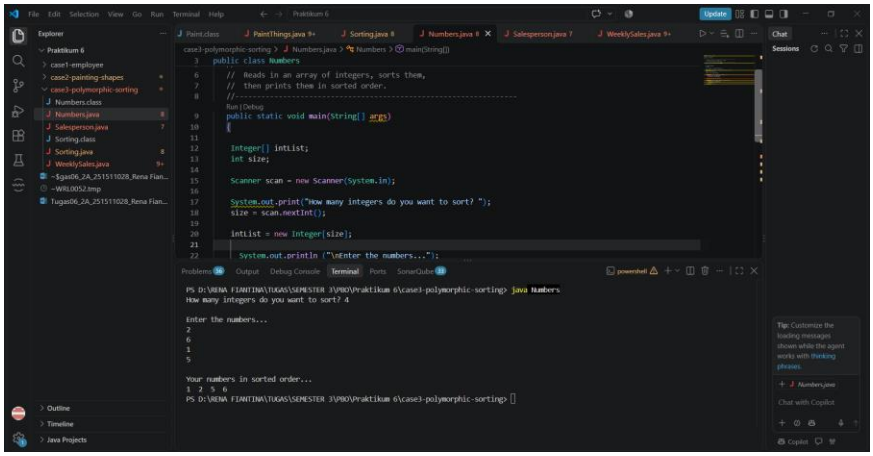
Method selectionSort() pada class Sorting menerima parameter berupa array bertipe Comparable[], sedangkan variabel intList pada class Numbers bertipe int[]. Kedua tipe tersebut tidak kompatibel karena int[] merupakan array tipe data primitif, sedangkan Comparable[] merupakan array objek yang memenuhi kontrak Comparable.

Alasan primitive int tidak memenuhi kontrak Comparable

Tipe data int digunakan untuk menyimpan angka biasa, sedangkan algoritma sorting pada praktikum ini membutuhkan objek yang bisa dibandingkan menggunakan method compareTo(). Karena itu, array int[] tidak dapat langsung digunakan. Solusinya adalah menggunakan Integer[], yaitu tipe objek untuk menyimpan angka yang dapat digunakan dalam proses sorting tersebut.

b. Penggunaan Integer[]

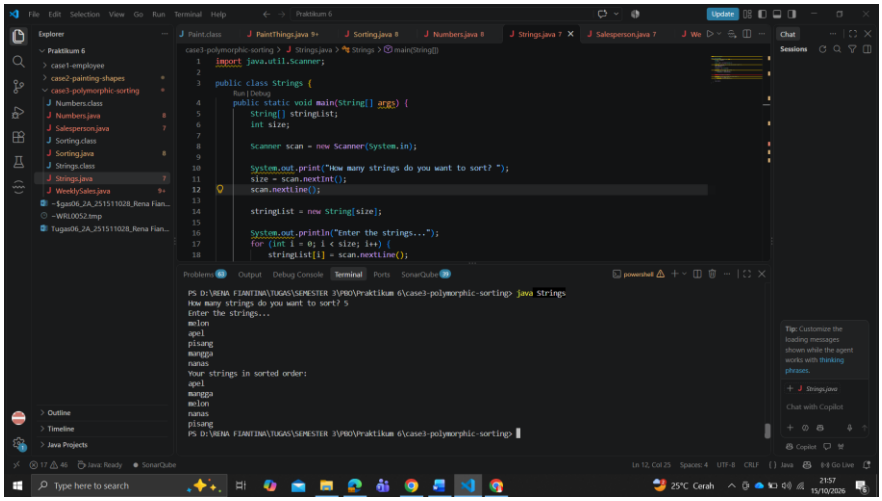
Mengubah array int[] menjadi Integer[] agar sesuai dengan kebutuhan method sorting yang menerima objek bertipe Comparable.



Program berhasil dikompilasi dan dijalankan menggunakan Integer[]. Angka yang dimasukkan dapat diproses oleh algoritma sorting tanpa error tipe seperti pada percobaan sebelumnya.

c. Pengurutan String

Menguji penggunaan algoritma sorting pada array String[] untuk menunjukkan bahwa algoritma yang sama dapat mengurutkan objek teks yang mendukung interface Comparable.



Program berhasil mengurutkan data String secara alfabet menggunakan method sorting yang ada pada class Sorting.

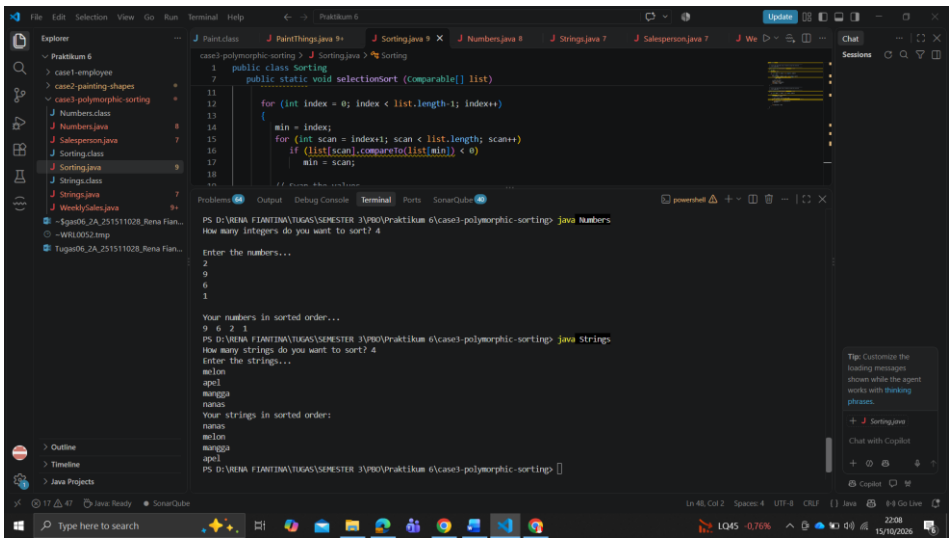
Checkpoint 8 : *Mengapa Sorting.selectionSort(...) dapat digunakan untuk Integer[] dan String[] walaupun kedua class tersebut tidak berada dalam satu hierarchy buatan Anda?*

⇨ Method Sorting.selectionSort() dapat digunakan untuk mengurutkan Integer[] dan String[] karena kedua class tersebut mendukung interface Comparable. Interface Comparable menyediakan method compareTo() yang digunakan untuk membandingkan objek.

Dengan begitu, Algoritma sorting cukup memanggil compareTo() untuk membandingkan elemen, sehingga algoritma yang sama dapat digunakan untuk kedua tipe objek meskipun keduanya tidak ada dalam satu hierarchy

d. Perubahan insertionSort() Menjadi Descending

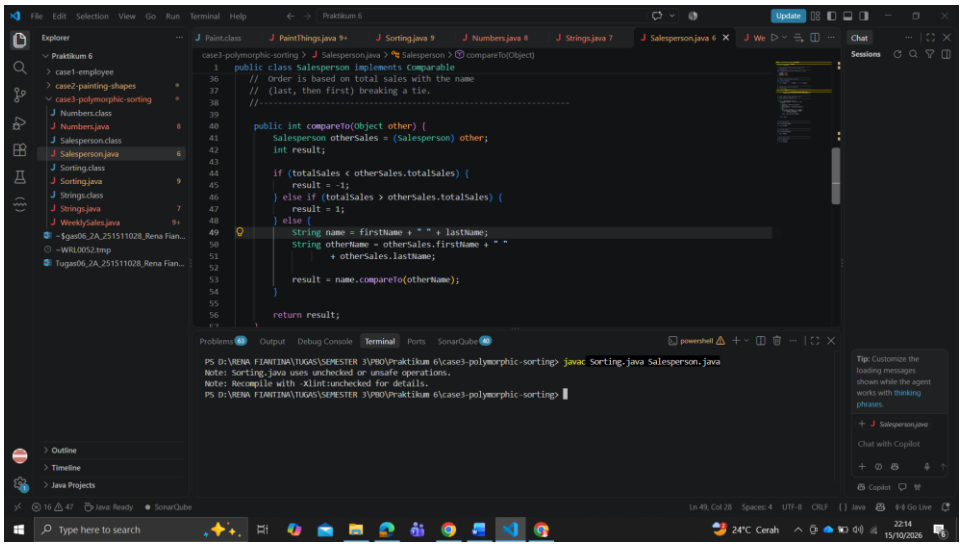
Mengubah kondisi perbandingan pada method insertionSort() agar elemen diurutkan dari nilai terbesar ke terkecil.



Program berhasil mengurutkan angka dan teks dalam urutan descending sesuai dengan aturan perbandingan yang diterapkan.

e. Implementasi Salesperson.compareTo()

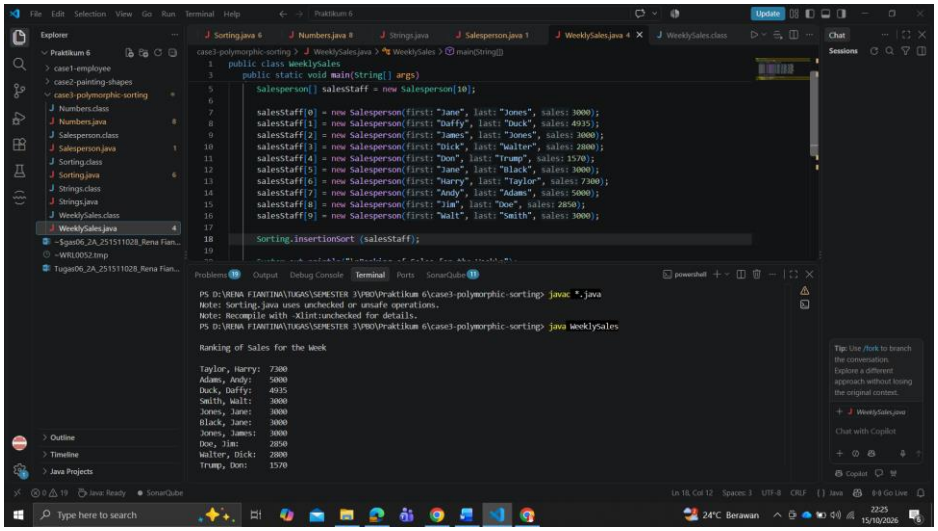
Mengimplementasikan method compareTo() pada class Salesperson untuk membandingkan objek berdasarkan total penjualan dan nama jika total penjualannya sama.



Class Salesperson dapat dibandingkan menggunakan method compareTo(), sehingga objek penjual dapat diproses oleh algoritma sorting yang menggunakan interface Comparable.

f. Pengujian WeeklySales

Menguji algoritma sorting pada kumpulan objek Salesperson untuk menampilkan peringkat penjualan mingguan berdasarkan total penjualan.



Program berhasil menampilkan peringkat sepuluh penjual dari total penjualan terbesar hingga terkecil. Penjual dengan total penjualan yang sama diurutkan berdasarkan nama secara terbalik alfabet, sesuai dengan compareTo() dan insertionSort() descending.